ML модель для выявления рисков при доставке

ML модель для выявления рисков при доставке

Цель:

Создать модель для выявления риска по недостатку времени при доставке для поиска путей оптимизации и минимизации затрат

Задача:

Построить ML модель классификации, которая по профилю заказчика выявляет вероятность неудачной доставки, чтобы минимизировать финансовые потери

План работы

- Этап 1: Загрузка данных и знакомство с ними
- Этап 2: EDA
- Этап 3: Создание выборок (GroupShuffleSplit)
- Этап 4: Обучение базовой модели
- Этап 5: Кластеризация по географическим признакам
- Этап 6: Кластеризация по профилям клиентов
- Этап 7: Обучение модели на новых признаках
- Этап 8: Тестирование лучшей модели
- Этап 9: Выводы и рекомендации для бизнеса

Перевод бизнес-задачи на язык машинного обучения

Бизнес задача: выявления риска по недостатку времени при доставке для поиска путей оптимизации и минимизации затрат Это задача бинарной классификации со значением 1 - времени на доставку не хватило, 0 - хватило времени на доставку.

Для решения задачи будут использованы модель градиентного бустинга CatBoost и логистической регрессии в качестве базовых моделей для проверки какая лучше всего описывает данные С помощью KMeans++ будет проведена кластеризация объектов, для проверки гипотезы улучшения качества модели при кластеризации

Технические метрики

- ROC-AUC - способность модели различать положительный и отрицательный классы независимо от выбранного порога классификации
- AUC-PR - площадь под кривой Precision-Recall. Чем ближе значение к 1, тем лучше модель разделяет классы на возможных порогах информативна на несбалансированных данных
- Precision - доля объектов, предсказанных моделью как положительный класс, которые действительно относятся к положительному классу
- Recall - доля объектов положительного класса, которые модель смогла найти

Этап 1 Подготовка среды: загрузка данных, фиксация random_state, установка библиотек

In [1]:
```
import numpy as np
import pandas as pd
import phik
import optuna
import time
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import (GroupShuffleSplit, cross_validate, cross_val_predict, cross_val_score, StratifiedKFold)
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, StandardScaler
from sklearn.impute import SimpleImputer

from sklearn.linear_model import LogisticRegression
import catboost as cb
from catboost import CatBoostClassifier
```

```
from sklearn.cluster import KMeans
from sklearn.manifold import TSNE


from sklearn.metrics import (accuracy_score, roc_auc_score, precision_score, recall_score, f1_score, precision_recall_curve, roc
_curve,
                             average_precision_score, confusion_matrix, ConfusionMatrixDisplay, classification_report, make_scor
er)


import shap
```

Зафиксируем `random_state` и параметры отображения

In [2]:
```
plt.style.use('default')
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 12

RANDOM_STATE = 42
```

- `plt.style.use('default')` - использование стандартного стиля визуализации matplotlib.
- `plt.rcParams['figure.figsize'] = (10, 6)` - размер графиков.
- `plt.rcParams['font.size'] = 12` - размер шрифта на графиках.
- `RANDOM_STATE = 42` - фиксация генератора случайных чисел для воспроизводимости результатов.

Загрузим данные из CSV-файлов:

- ds_s18_customers.csv
- ds_s18_orders.csv

In [3]:
```
try:
    customers = pd.read_csv('ds_s18_customers.csv')
    orders = pd.read_csv('ds_s18_orders.csv')
except:
    customers = pd.read_csv('https://code.s3.yandex.net/datasets/ds_s18_customers.csv')
    orders = pd.read_csv('https://code.s3.yandex.net/datasets/ds_s18_orders.csv')
```

In [4]:
```
datasets = {'customers': customers,
            'orders': orders,}

for name, df in datasets.items():
    print(f'{name}: {df.shape}')
```

Out:
```
customers: (20652, 8)
orders: (180519, 7)
```

Среда разработки поготовлена, данные выгружены. Переходим к EDA

Этап 2: EDA

In [5]:
```
def datasets_info(datasets):
    for name, df in datasets.items():
        print(name)

        for col in df.columns:
            print(f"\t{col}"
                  f"\n\t\tтип: {df[col].dtype}"
                  f"\n\t\tвсего строк: {len(df[col])}"
```

```
                f"\n\t\tпропусков: {df[col].isna().sum()}"
                f"\n\t\tуникальных значений: {len(list(df[col].dropna().unique()))}"
                f"\n\t\tпримеры возможных значений: {list(df[col].dropna().unique()[:5])}")
        print()
datasets_info(datasets)
```

Out:

```
customers
        customer_id
                тип: int64
                всего строк: 20652
                пропусков: 0
                уникальных значений: 20652
                примеры возможных значений: [np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5)]
        customer_lat
                тип: float64
                всего строк: 20652
                пропусков: 0
                уникальных значений: 12769
                примеры возможных значений: [np.float64(25.95364761), np.float64(38.37559509), np.float64(18.025375370000003), np.float64(3
3.67002106), np.float64(18.35910416)]
        customer_lon
                тип: float64
                всего строк: 20652
                пропусков: 0
                уникальных значений: 6161
                примеры возможных значений: [np.float64(-97.5076828), np.float64(-104.7260208), np.float64(-66.61508178999999), np.float64(-
112.2470779), np.float64(-66.07791138)]
        total_sales
                тип: float64
                всего строк: 20652
                пропусков: 0
                уникальных значений: 12632
                примеры возможных значений: [np.float64(472.4500122), np.float64(1618.6600417900004), np.float64(3189.2000370200003), np.flo
at64(1480.70999322), np.float64(1101.91999814)]
        total_orders
                тип: int64
                всего строк: 20652
                пропусков: 0
                уникальных значений: 15
                примеры возможных значений: [np.int64(1), np.int64(4), np.int64(5), np.int64(3), np.int64(7)]
        avg_discount
                тип: float64
                всего строк: 20652
                пропусков: 0
                уникальных значений: 10600
                примеры возможных значений: [np.float64(0.059999999), np.float64(0.1260000006), np.float64(0.10500000066666666), np.float64
(0.1350000006428571), np.float64(0.1400000019999999)]
        return_rate
                тип: float64
                всего строк: 20652
                пропусков: 0
                уникальных значений: 166
                примеры возможных значений: [np.float64(0.0), np.float64(0.1176470588235294), np.float64(0.2), np.float64(0.15), np.float64
(0.3333333333333333)]
        recency
                тип: int64
                всего строк: 20652
                пропусков: 0
                уникальных значений: 1012
                примеры возможных значений: [np.int64(793), np.int64(137), np.int64(230), np.int64(381), np.int64(458)]

orders
        order_id
                тип: int64
                всего строк: 180519
                пропусков: 0
                уникальных значений: 180519
                примеры возможных значений: [np.int64(180517), np.int64(179254), np.int64(179253), np.int64(179252), np.int64(179251)]
        customer_id
                тип: int64
                всего строк: 180519
                пропусков: 0
```

```
                уникальных значений: 20652
                примеры возможных значений: [np.int64(20755), np.int64(19492), np.int64(19491), np.int64(19490), np.int64(19489)]
        late_delivery_risk
                тип: int64
                всего строк: 180519
                пропусков: 0
                уникальных значений: 2
                примеры возможных значений: [np.int64(0), np.int64(1)]
        shipping_mode
                тип: str
                всего строк: 180519
                пропусков: 0
                уникальных значений: 4
                примеры возможных значений: ['Standard Class', 'First Class', 'Second Class', 'Same Day']
        category_name
                тип: str
                всего строк: 180519
                пропусков: 0
                уникальных значений: 50
                примеры возможных значений: ['Sporting Goods', 'Cleats', 'Shop By Sport', "Women's Apparel", 'Electronics']
        order_date
                тип: str
                всего строк: 180519
                пропусков: 0
                уникальных значений: 65752
                примеры возможных значений: ['2018-01-31 22:56:00', '2018-01-13 12:27:00', '2018-01-13 12:06:00', '2018-01-13 11:45:00', '20
18-01-13 11:24:00']
        item_price
                тип: float64
                всего строк: 180519
                пропусков: 0
                уникальных значений: 75
                примеры возможных значений: [np.float64(327.75), np.float64(59.99000168), np.float64(39.99000168), np.float64(50.0), np.floa
t64(44.99000168)]
```

Вывод по первичному анализу

- customers всего строк 20652
  - `customer_id` количество строк совпадает с количеством уникальным значений все идентефекаторы уникальны
  - `customer_lat` `customer_lon` признаки на месторасположения, в совокупности даю удаленность клиента можно использовать для подсчета евклидово расстояния, следует провести проверку на нестандартные значения с точки зрения географии
  - `total_sales` `total_orders` `recency` основные признаки для составления портрета пользователя следует сделать проверку на выбросы и проанализировать распределение
  - `return_rate` `avg_discount` признаки помогающие дополнить профиль пользователя так же как и с основными следует сделать проверку на выбросы и проанализировать распределение
  - Явных пропусков и возможных дубликатов id нет, следует сделать проверку на невалидные значения и возможные значения заглушки для пропусков
- orders всего строк 180519
  - `order_id` количество строк совпадает с количеством уникальным значений все идентефекаторы уникальны
  - `customer_id` количество уникальных значений совпадает с количеством в `customers` следует явно проверить что идентификаторы в обоих таблицах одинаковы
  - `late_delivery_risk` целевая переменная созначениями 1 и 0 аномалий нет следует проверить дисбаланс
  - `shipping_mode` значение соответствуют заявленным следует рассмотреть распределение целевой переменной в зависимоти от режима доставки
  - `category_name` ассортимент велик 50 разных категорий следует рассмотреть есть ли аномальное количество неудач в одной из них
  - `order_date` хранит дату и время доставки необходима перетипизация и разбиение по отдельным переменным месяц год время после можно рассмотреть менялось ли количество неудачных доставок с изменением даты, а так же рассмотреть пики заказов по времени
  - `item_price` можно рассмотреть вероятность неудачи в зависимости от цены товара, так как у нас нет признака габарита, мы можем лтшь догадываться что он зависит от стоимости

Проверка на дубликаты

In [6]:
```
id_cols = ['customer_id', 'order_id']
def duplicates_report(df, id_cols=[]):
    full_dup = df.duplicated().sum()
    print("Полные дубликаты строк:", full_dup)

    if id_cols:
        for id_col in id_cols:
            if id_col in df.columns:
                id_dup = df.duplicated(subset=[id_col]).sum()
                print(f"Дубликаты по {id_col}: {id_dup}")
                no_id = df.drop(columns=[id_col])
                no_id_dup = no_id.duplicated().sum()
                print(f"Дубликаты без {id_col}: {no_id_dup}")

for name, df in datasets.items():
    print(name)
    duplicates_report(df, id_cols)
    print("\n")
```

Out:
```
customers
Полные дубликаты строк: 0
Дубликаты по customer_id: 0
Дубликаты без customer_id: 5


orders
Полные дубликаты строк: 0
Дубликаты по customer_id: 159867
Дубликаты без customer_id: 0
Дубликаты по order_id: 0
Дубликаты без order_id: 20806
```

Дубликаты в orders без order_id: 20806: один клиент не может совершить два идентичных заказа в один момент времени Дубликаты в customers без id: два разных клиента не могут иметь одинаковые координаты, продажи и поведение записей. Удаляем дубликаты, сохраняя первое вхождение

In [7]:
```
mask = customers.drop(columns=['customer_id']).duplicated(keep='first')
customers = customers[~mask]
mask = orders.drop(columns=['order_id']).duplicated(keep='first')
orders = orders[~mask]

datasets = {'customers': customers,
            'orders': orders}
for name, df in datasets.items():
    print(name)
    duplicates_report(df, id_cols)
    print("\n")
```

Out:
```
customers
Полные дубликаты строк: 0
Дубликаты по customer_id: 0
Дубликаты без customer_id: 0


orders
Полные дубликаты строк: 0
Дубликаты по customer_id: 139061
Дубликаты без customer_id: 0
Дубликаты по order_id: 0
Дубликаты без order_id: 0
```

Разобьём order_date на временные столбцы `order_month`, `order_weekday`, `order_hour`

In [8]:
```
orders['order_date'] = pd.to_datetime(orders['order_date'])
orders['order_month'] = orders['order_date'].dt.month
orders['order_weekday'] = orders['order_date'].dt.dayofweek
orders['order_hour'] = orders['order_date'].dt.hour
orders.head(5)
```

Out:

	order_id	customer_id	late_delivery_risk	shipping_mode	category_name	order_date	item_price	order_month	order_weekday	orde
0	180517	20755	0	Standard Class	Sporting Goods	2018-01-31 22:56:00	327.75	1	2	22
1	179254	19492	1	Standard Class	Sporting Goods	2018-01-13 12:27:00	327.75	1	5	12
2	179253	19491	0	Standard Class	Sporting Goods	2018-01-13 12:06:00	327.75	1	5	12
3	179252	19490	0	Standard Class	Sporting Goods	2018-01-13 11:45:00	327.75	1	5	11
4	179251	19489	0	Standard Class	Sporting Goods	2018-01-13 11:24:00	327.75	1	5	11

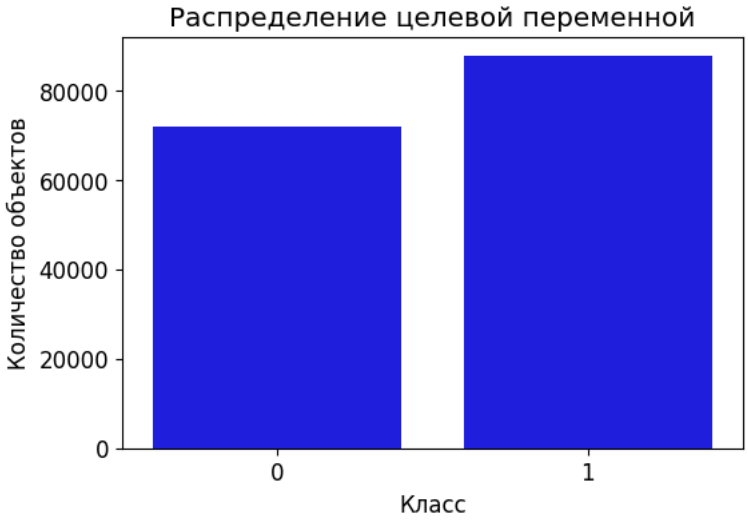
Проверим баланс классов

In [9]:
```
def target_distribution(x):
    value_counts = x.value_counts()
    value_share = x.value_counts(normalize=True)
    print('Количество объектов:')
    print(value_counts)
    print('Доли классов:')
    print(value_share)

    plt.figure(figsize=(6, 4))
    sns.countplot(x=x, color='blue')
    plt.title('Распределение целевой переменной')
    plt.xlabel('Класс')
    plt.ylabel('Количество объектов')
    plt.show()
target_distribution(datasets['orders']['late_delivery_risk'])
```

Out:

```
Количество объектов:
late_delivery_risk
1    87653
0    72060
Name: count, dtype: int64
Доли классов:
late_delivery_risk
1    0.548816
0    0.451184
Name: proportion, dtype: float64
```



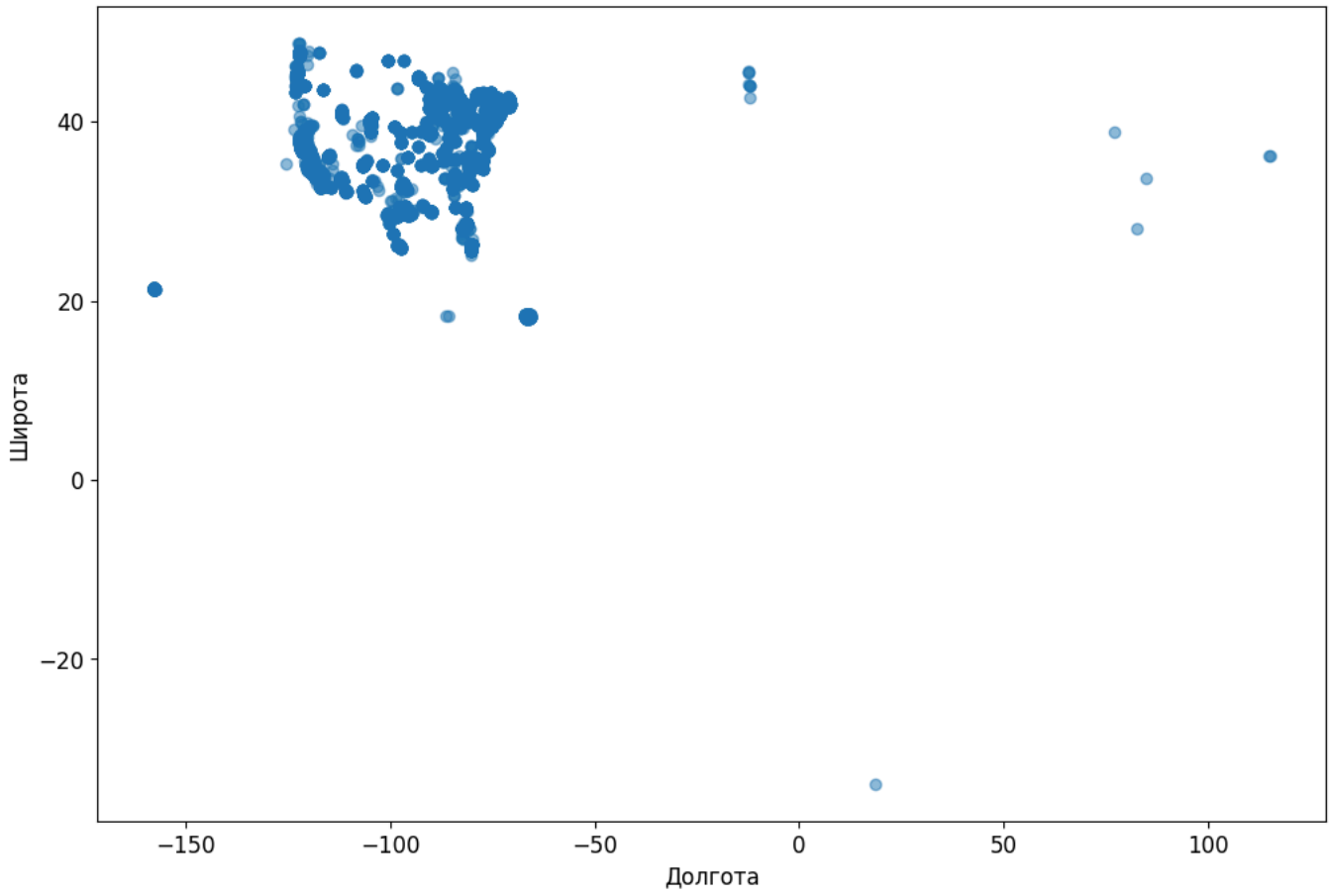
Дисбаланс классов минимален

Подтверждено, что около 55% всех заказов доставляются с задержкой

Географический аудит

In [10]:
```
plt.figure(figsize=(12, 8))
plt.scatter(customers['customer_lon'], customers['customer_lat'], alpha=0.5)
plt.xlabel('Долгота')
plt.ylabel('Широта')
plt.show()
```

Out:



обнаружены аномальные точки, удаляем согласно заданным условиям по координатам ""*значения, выходящие за пределы диапазона долготы −125 до −20*""

In [11]:
```
print("До:")
print("Клиентов:", len(customers))
print("Заказов:", len(orders))

customers = customers[(customers['customer_lon'] >= -125) & (customers['customer_lon'] <= -20)]
orders = orders[orders['customer_id'].isin(customers['customer_id'])]

print("После:")
print("Клиентов:", len(customers))
print("Заказов:", len(orders))
```

Out:

```
До:
Клиентов: 20647
Заказов: 159713
После:
Клиентов: 20491
Заказов: 158444
```

Обработка категориальный значений

In [12]:
```
def categorical_analysis(df, cat_cols):
    for col in cat_cols:
        print("Столбец:", col)
        print("Уникальных значений:", df[col].nunique())
        print("Примеры значений:", df[col].unique()[:10])
        print()

cat_cols = ['shipping_mode', 'category_name']

for name, df in datasets.items():
    cols = [col for col in cat_cols if col in df.columns]

    if cols:
        print(name, "\n")
        categorical_analysis(df, cols)
        print("\n")
```

Out:

```
orders

Столбец: shipping_mode
Уникальных значений: 4
Примеры значений: <StringArray>
['Standard Class', 'First Class', 'Second Class', 'Same Day']
Length: 4, dtype: str

Столбец: category_name
Уникальных значений: 50
Примеры значений: <StringArray>
[    'Sporting Goods',            'Cleats',      'Shop By Sport',
    'Women's Apparel',       'Electronics',       'Boxing & MMA',
   'Cardio Equipment',          'Trade-In',   'Kids' Golf Clubs',
 'Hunting & Shooting']
Length: 10, dtype: str
```

странных значений нет ни у одного признака, для кодировки признаков используем

- OHE для `shipping_mode`
- Ordinal Encoding для category_name

Анализ выбросов и распределления

In [13]:
```
def data_review(df, col):
    data = df[col].dropna()
    print(data.describe())

    q1 = data.quantile(0.25)
    q3 = data.quantile(0.75)
    iqr = q3 - q1
    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr

    outliers = data[(data < lower_bound) | (data > upper_bound)]
    high_outliers = data[data > upper_bound].sort_values(ascending=True)
    low_outliers = data[data < lower_bound].sort_values(ascending=True)

    print("Выбросы:")
    print("Количество:", len(outliers))
    print("Процент:", round(len(outliers) / len(data) * 100, 2))
    print("Границы:", round(lower_bound, 2), round(upper_bound, 2))
    print("Первые 5 выбросов выше верхней границы:", high_outliers.head(5).values)
    print("5 максимальных значений:", high_outliers.tail(5).values)
    print("Первые 5 выбросов ниже нижней границы:", low_outliers.tail(5).values)
    print("5 минимальных значений:", low_outliers.head(5).values)

    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    sns.histplot(data, kde=True, ax=axes[0])
    axes[0].set_title(f"Hist: {col}")
    sns.boxplot(x=data, ax=axes[1])
    axes[1].set_title(f"Boxplot: {col}")
    plt.tight_layout()
    plt.show()
```

In [14]:
```
num_cols = ['item_price', 'order_hour', 'order_weekday',
            'order_month', 'total_sales', 'total_orders', 'recency', 'avg_discount', 'return_rate']

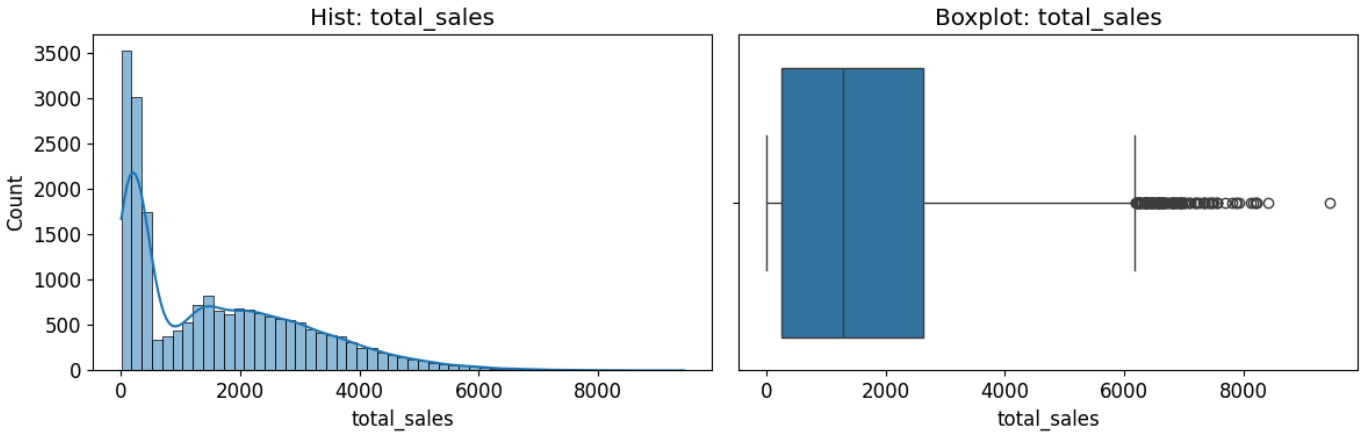
for name, df in datasets.items():
    cols = [col for col in num_cols if col in df.columns]
    if cols:
        print("\n\n", name, "\n")
        for col in cols:
            print("Столбец: ", col, "\n")
            data_review(df, col)
```

Out:

```
 customers

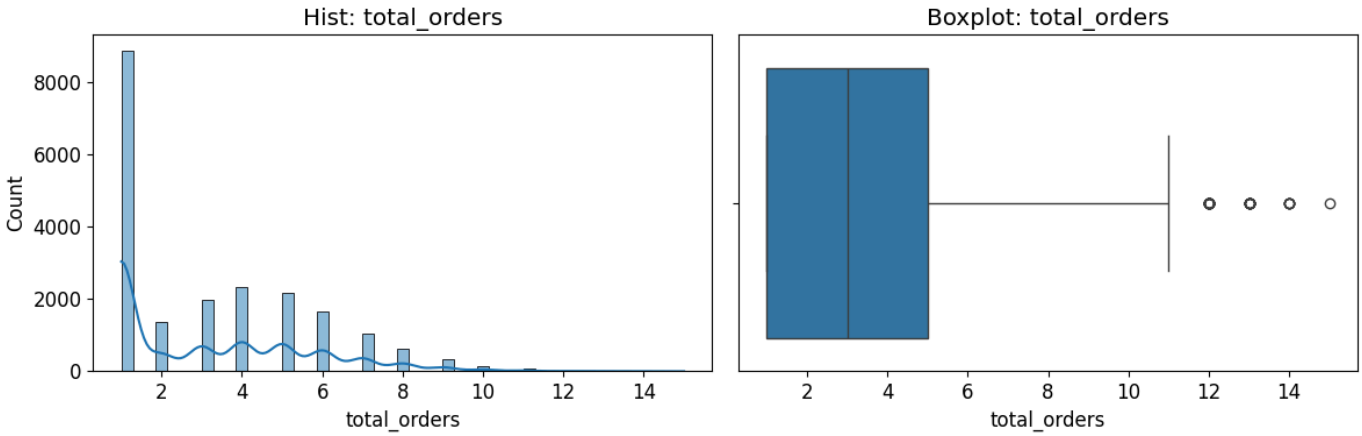
Столбец:  total_sales

count    20647.000000
mean      1600.880648
std       1508.441813
min          8.470000
25%        254.940002
50%       1295.219995
75%       2621.685043
max       9436.610088
Name: total_sales, dtype: float64
Выбросы:
Количество: 122
Процент: 0.59
Границы: -3295.18 6171.8
Первые 5 выбросов выше верхней границы: [6188.54007722 6194.66010668 6200.46009836 6206.85003289 6207.5001184 ]
5 максимальных значений: [8198.15006854 8221.64012954 8222.670103   8400.98010666 9436.61008833]
Первые 5 выбросов ниже нижней границы: []
5 минимальных значений: []
```




```
Столбец:  total_orders

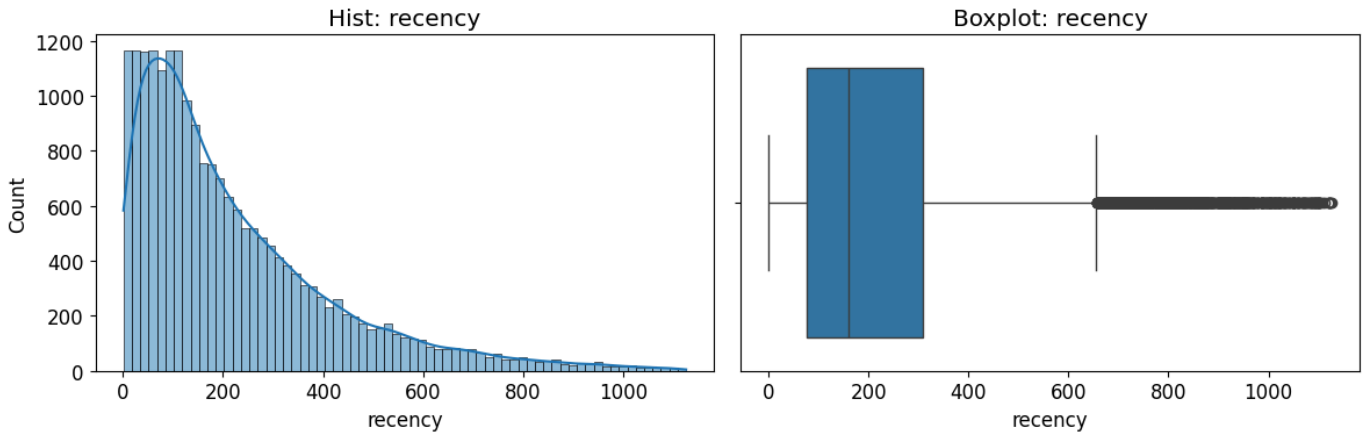
count    20647.000000
mean         3.184337
std          2.430756
min          1.000000
25%          1.000000
50%          3.000000
75%          5.000000
max         15.000000
Name: total_orders, dtype: float64
Выбросы:
Количество: 47
Процент: 0.23
Границы: -5.0 11.0
Первые 5 выбросов выше верхней границы: [12 12 12 12 12]
5 максимальных значений: [14 14 14 14 15]
Первые 5 выбросов ниже нижней границы: []
5 минимальных значений: []
```




```
Столбец:  recency

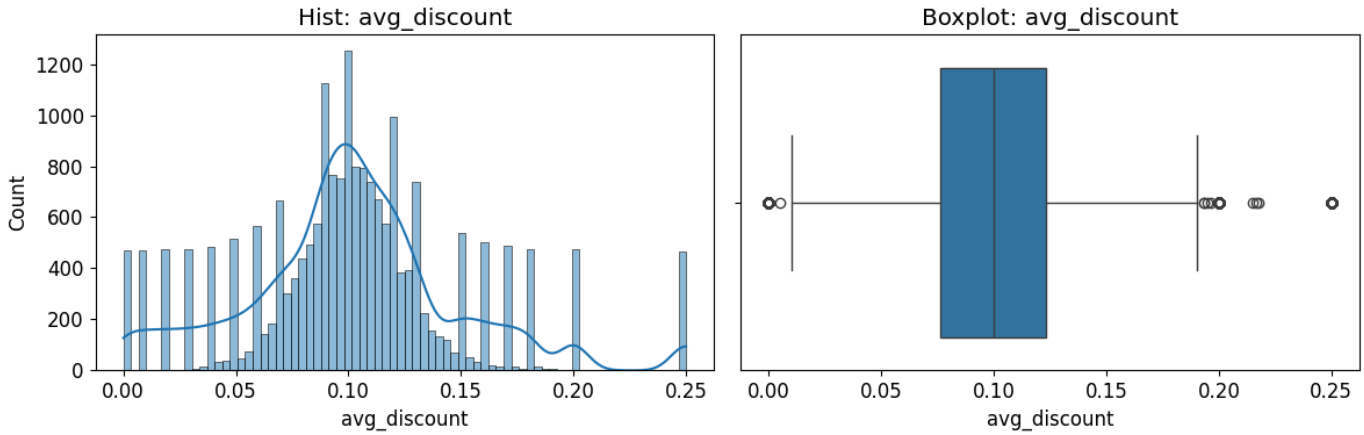
count    20647.000000
mean       221.129026
std        199.402272
min          1.000000
25%         76.000000
50%        160.000000
75%        308.000000
max       1126.000000
Name: recency, dtype: float64
Выбросы:
Количество: 951
Процент: 4.61
Границы: -272.0 656.0
Первые 5 выбросов выше верхней границы: [657 657 657 657 657]
5 максимальных значений: [1121 1122 1123 1124 1126]
```

```
Первые 5 выбросов ниже нижней границы: []
5 минимальных значений: []
```




```
Столбец:  avg_discount

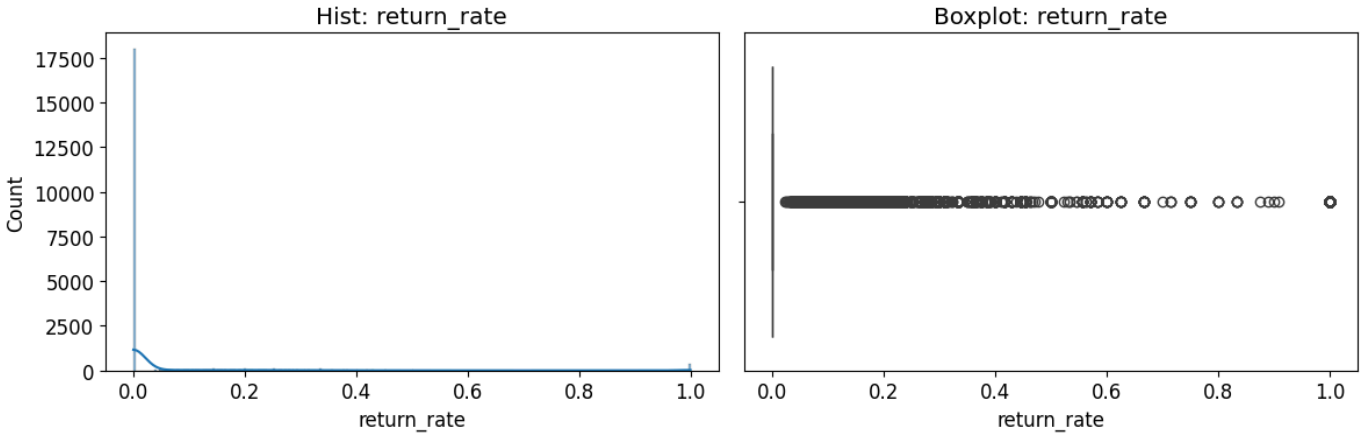
count    20647.000000
mean         0.101702
std          0.048254
min          0.000000
25%          0.076250
50%          0.100000
75%          0.123000
max          0.250000
Name: avg_discount, dtype: float64
Выбросы:
Количество: 1417
Процент: 6.86
Границы: 0.01 0.19
Первые 5 выбросов выше верхней границы: [0.19333333 0.19333333 0.195      0.19666667 0.2       ]
5 максимальных значений: [0.25 0.25 0.25 0.25 0.25]
Первые 5 выбросов ниже нижней границы: [0.    0.    0.    0.    0.005]
5 минимальных значений: [0. 0. 0. 0. 0.]
```




```
Столбец:  return_rate

count    20647.000000
mean         0.042977
std          0.156624
min          0.000000
25%          0.000000
50%          0.000000
75%          0.000000
max          1.000000
Name: return_rate, dtype: float64
Выбросы:
Количество: 2633
Процент: 12.75
Границы: 0.0 0.0
```

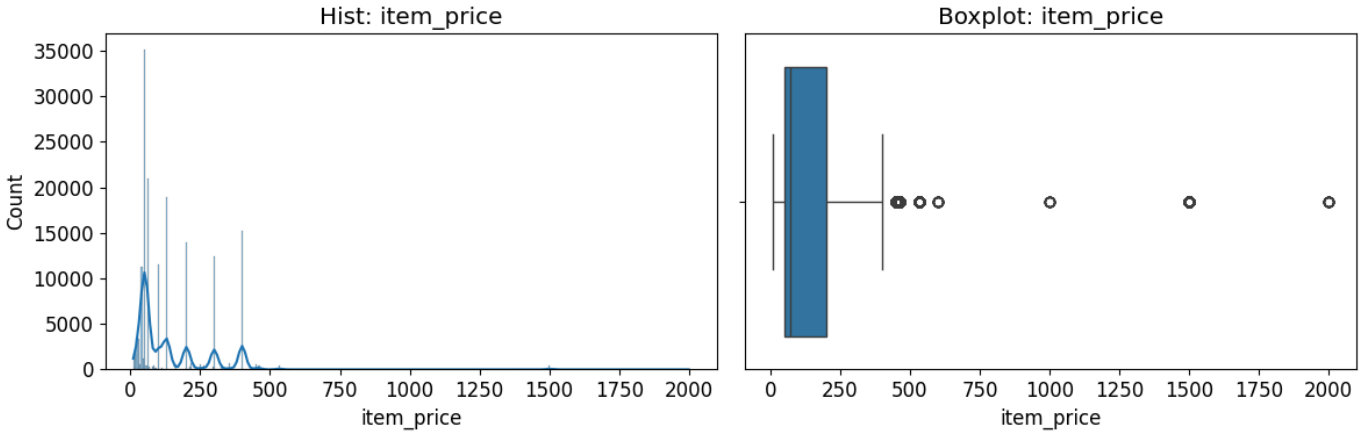
```
Первые 5 выбросов выше верхней границы: [0.02222222 0.02380952 0.02631579 0.02857143 0.02941176]
5 максимальных значений: [1. 1. 1. 1. 1.]
Первые 5 выбросов ниже нижней границы: []
5 минимальных значений: []
```




```
 orders

Столбец:  item_price

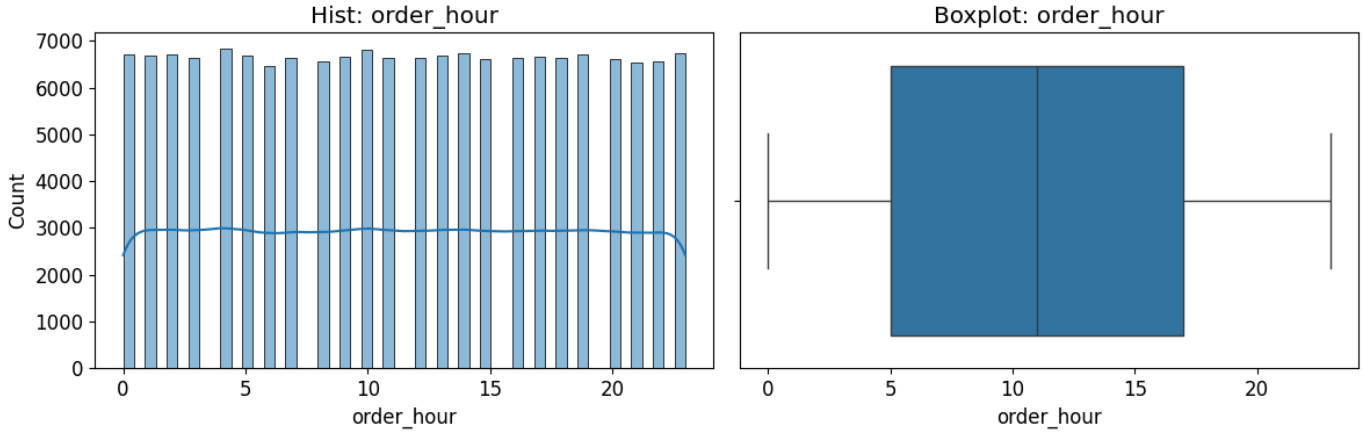
count    159713.000000
mean        142.180390
std         142.631690
min           9.990000
25%          50.000000
50%          70.000000
75%         199.990005
max        1999.989990
Name: item_price, dtype: float64
Выбросы:
Количество: 2045
Процент: 1.28
Границы: -174.99 424.98
Первые 5 выбросов выше верхней границы: [452.0400085 452.0400085 452.0400085 452.0400085 452.0400085]
5 максимальных значений: [1999.98999 1999.98999 1999.98999 1999.98999 1999.98999]
Первые 5 выбросов ниже нижней границы: []
5 минимальных значений: []
```




```
Столбец:  order_hour

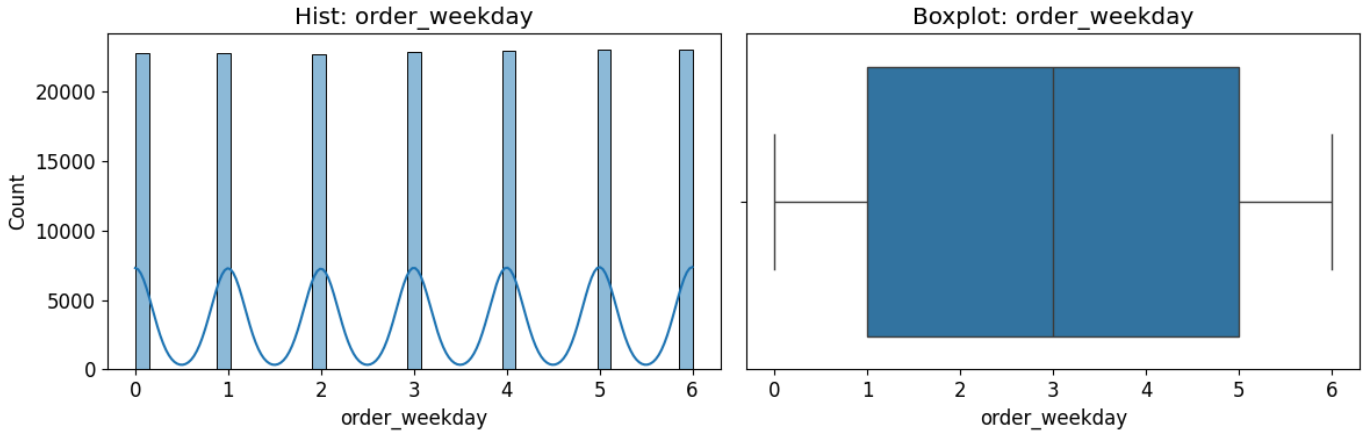
count    159713.000000
mean         11.481401
std           6.925099
min           0.000000
25%           5.000000
50%          11.000000
75%          17.000000
max          23.000000
```

```
Name: order_hour, dtype: float64
Выбросы:
Количество: 0
Процент: 0.0
Границы: -13.0 35.0
Первые 5 выбросов выше верхней границы: []
5 максимальных значений: []
Первые 5 выбросов ниже нижней границы: []
5 минимальных значений: []
```




```
Столбец:  order_weekday

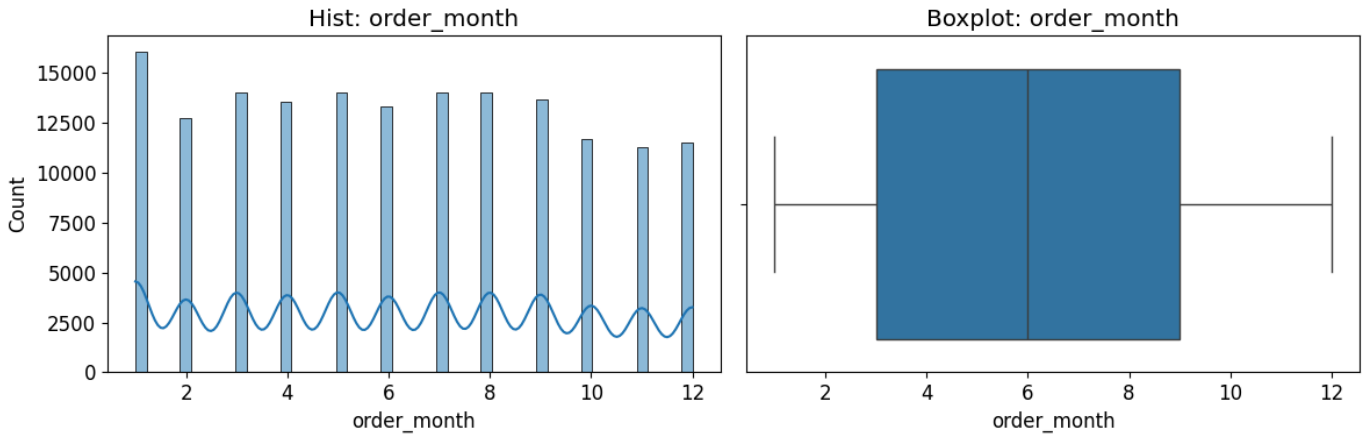
count    159713.000000
mean          3.008709
std           2.001575
min           0.000000
25%           1.000000
50%           3.000000
75%           5.000000
max           6.000000
Name: order_weekday, dtype: float64
Выбросы:
Количество: 0
Процент: 0.0
Границы: -5.0 11.0
Первые 5 выбросов выше верхней границы: []
5 максимальных значений: []
Первые 5 выбросов ниже нижней границы: []
5 минимальных значений: []
```




```
Столбец:  order_month

count    159713.000000
mean          6.254244
std           3.415452
min           1.000000
25%           3.000000
50%           6.000000
```

```
75%           9.000000
max          12.000000
Name: order_month, dtype: float64
Выбросы:
Количество: 0
Процент: 0.0
Границы: -6.0 18.0
Первые 5 выбросов выше верхней границы: []
5 максимальных значений: []
Первые 5 выбросов ниже нижней границы: []
5 минимальных значений: []
```



Проверка на мультиколлинеарность признаков

In [15]:
```
def phik_corr_matrix(df, interval_cols=None, drop_cols=None):
    data = df.copy()

    if drop_cols:
        data = data.drop(columns=drop_cols, errors='ignore')

    plt.figure(figsize = (8, 6))
    corr = data.phik_matrix(interval_cols=interval_cols)
    display(corr)
    sns.heatmap(corr, cmap='coolwarm', annot=True, fmt='.2f')
    plt.show()

    return corr
```

In [16]:
```
drop_cols = ['customer_id', 'order_id', 'order_date']
interval_cols = ['item_price', 'total_sales', 'total_orders', 'recency', 'avg_discount', 'return_rate', 'customer_lat', 'custom
er_lon']

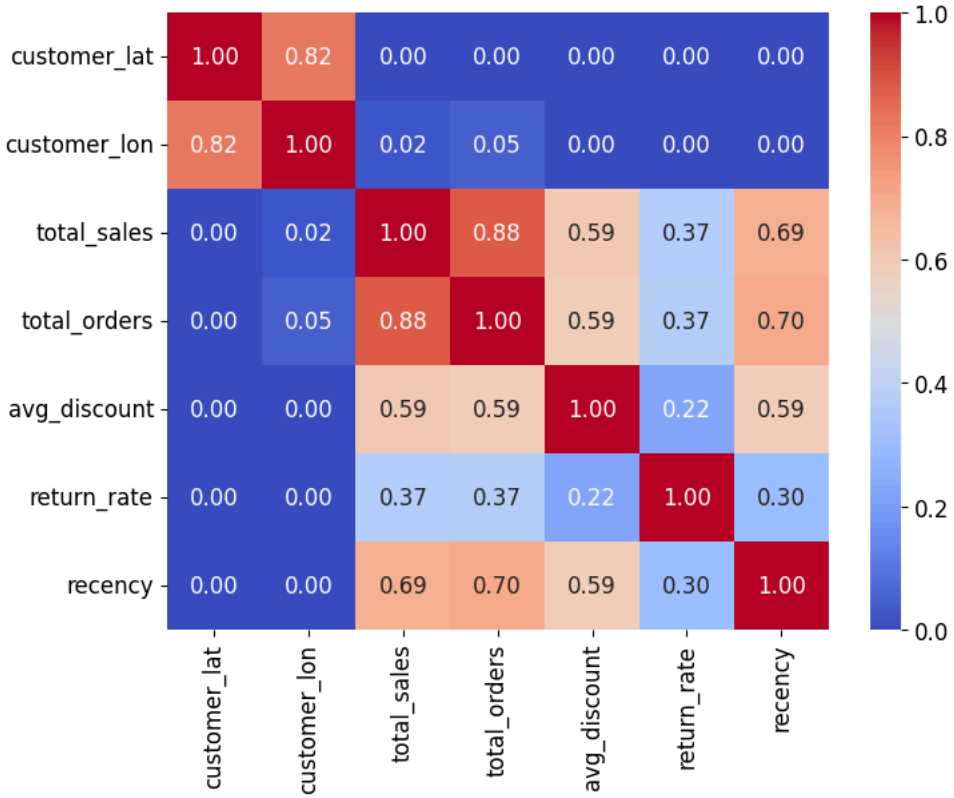
for name, df in datasets.items():
    print('Корреляции в', name)
    phik_corr_matrix(df, interval_cols, drop_cols)
    print()
```

Out:

```
Корреляции в customers
```

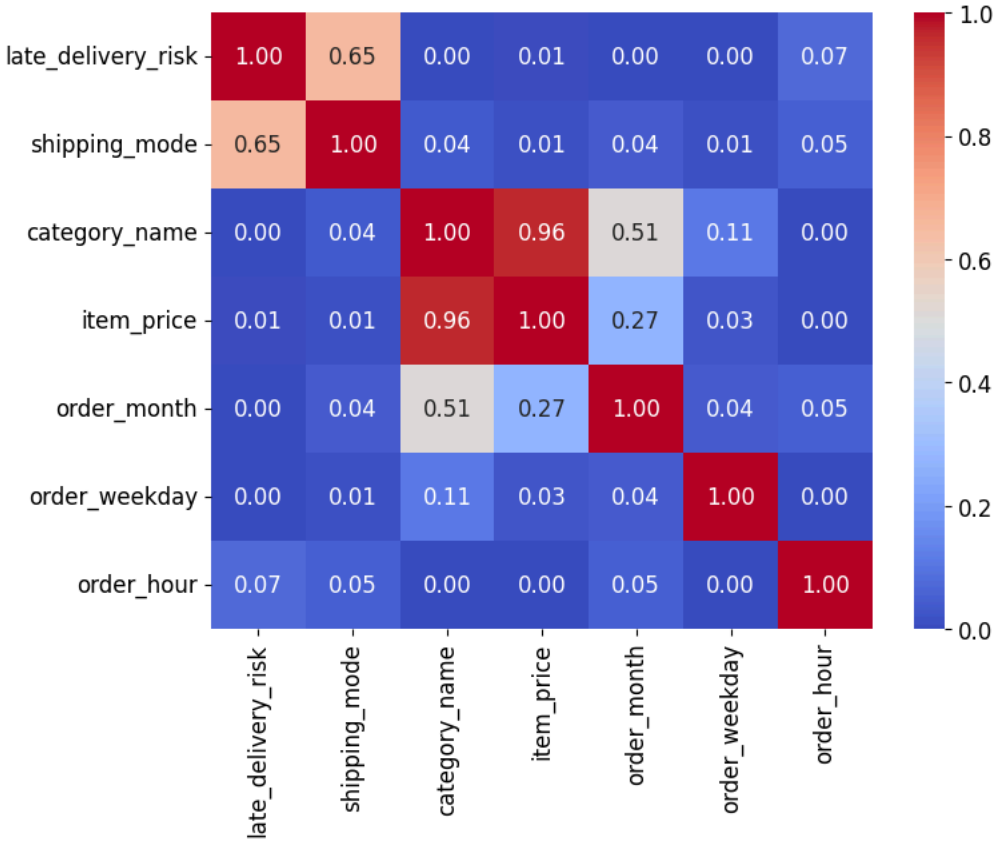
	customer_lat	customer_lon	total_sales	total_orders	avg_discount	return_rate	recency
customer_lat	1.000000	0.821802	0.000000	0.000000	0.000000	0.000000	0.000000
customer_lon	0.821802	1.000000	0.021499	0.046646	0.000000	0.000000	0.000000
total_sales	0.000000	0.021499	1.000000	0.880510	0.593832	0.367600	0.692487
total_orders	0.000000	0.046646	0.880510	1.000000	0.589426	0.372280	0.697123
avg_discount	0.000000	0.000000	0.593832	0.589426	1.000000	0.223029	0.591160

	customer_lat	customer_lon	total_sales	total_orders	avg_discount	return_rate	recency
return_rate	0.000000	0.000000	0.367600	0.372280	0.223029	1.000000	0.301557
recency	0.000000	0.000000	0.692487	0.697123	0.591160	0.301557	1.000000



> Корреляции в orders

	late_delivery_risk	shipping_mode	category_name	item_price	order_month	order_weekday	order_hour
late_delivery_risk	1.000000	0.653162	0.000000	0.005765	0.002948	0.002303	0.074860
shipping_mode	0.653162	1.000000	0.036811	0.012652	0.040153	0.008057	0.050112
category_name	0.000000	0.036811	1.000000	0.961668	0.508940	0.113089	0.000000
item_price	0.005765	0.012652	0.961668	1.000000	0.272813	0.030561	0.000000
order_month	0.002948	0.040153	0.508940	0.272813	1.000000	0.039651	0.052489
order_weekday	0.002303	0.008057	0.113089	0.030561	0.039651	1.000000	0.003865
order_hour	0.074860	0.050112	0.000000	0.000000	0.052489	0.003865	1.000000



Кореляции в customers

- `customer_lat` и `customer_lon` сильно коррелируют между собой это связано с клстерами которые образуют точки по широте и долготе
- `total_sales` сильная связь в `total_orders` чем больше заказов тем больше клиент оставляет денег связь обоснована
- `avg_discount` умеренная связь с `total_sales` , `total_orders` и `recency` связи могут быть вызваны бизнес стратегией компании с целью либо на удержание клиента, либо на привлечения новых проверим это рассмотрев как изменяется `avg_discount` при изменение `total_orders`
- `return_rate` умеренная связь с `total_sales` , `total_orders` и `recency` есть умеренная связь вероятности возврата с профилем клиента есть обоснование для дальнейшей кластеризации клиентов
- `recency` сильная связь с `total_sales` , `total_orders` и умеренная связ с `recency` количество неактивных дней клиента зависит от его активности, проверим зависимость `recency` от `total_orders`

Корреляции в orders

- `late_delivery_risk` и `shipping_mode` сильная связь типы доставки имеют разный риск задержки, проверим распределение целевой переменной по типам доставки
- `category_name` и `item_price` сильная связь разные категории товаров имеют разные ценовые диапазоны связь объясняется природой бизнеса, с `order_month` умеренная связь имеет место сезонность, в разные месяцы популярны разные категории товаров проверим: какие категории заказывают чаще в определённые месяцы
- Признаки времени и дня недели слабо связаня с целевой переменной

Все связи между признаками обоснованы природой данных и имеют обоснование с точки зрения бизнеса

In [17]:
```
print("Распределение целевой переменной по способам доставки")
sns.countplot(data=orders, x='shipping_mode', hue='late_delivery_risk')
plt.xlabel('Тип доставки')
plt.ylabel('Количество')
plt.show()

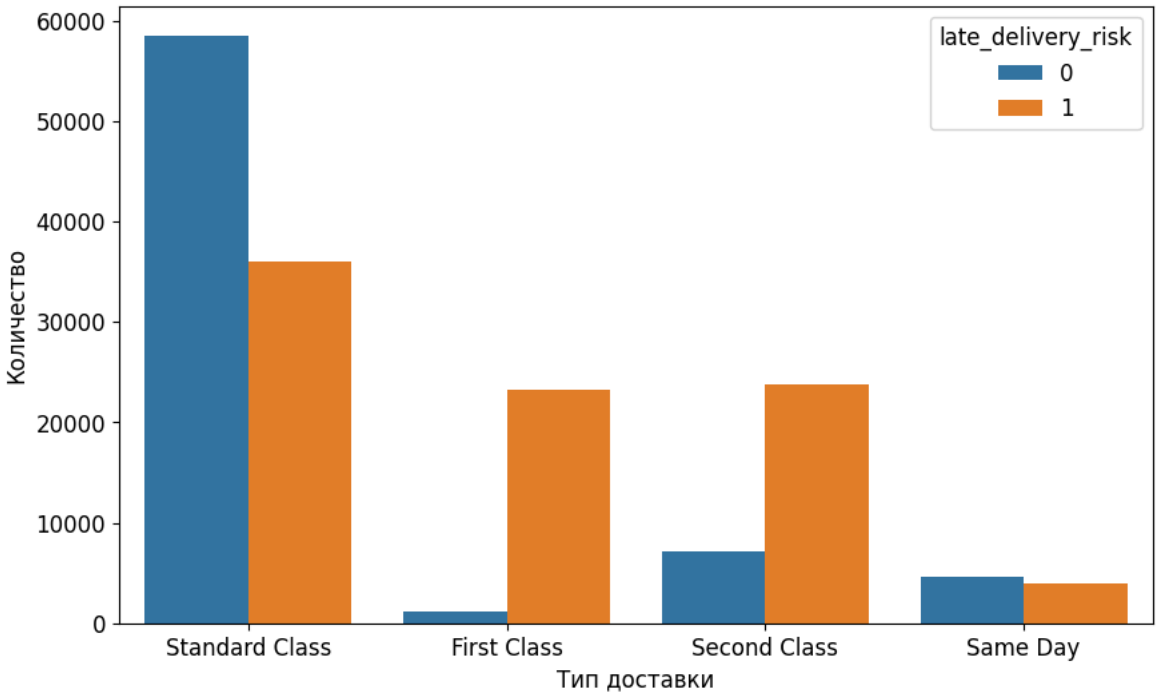
print('avg_discount при изменении total_orders')
avg_by_orders = customers.groupby('total_orders')['avg_discount'].mean().reset_index()
plt.plot(avg_by_orders['total_orders'], avg_by_orders['avg_discount'], color='darkgreen')
plt.xlabel('Количество заказов')
plt.ylabel('Средняя скидка')
```

```
plt.show()

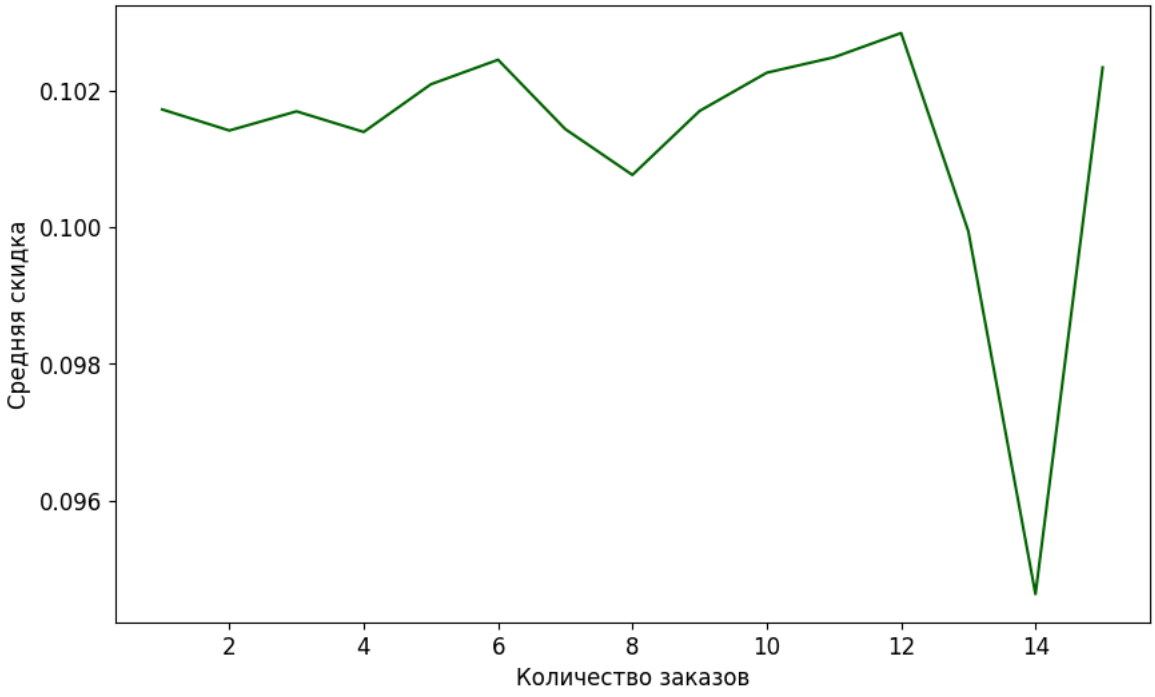
print('recency при изменении total_orders')
avg_recency = customers.groupby('total_orders')['recency'].mean().reset_index()
plt.plot(avg_recency['total_orders'], avg_recency['recency'], color='lightgreen')
plt.xlabel('Количество заказов')
plt.ylabel('Неактивные дни')
plt.show()
```

Out:

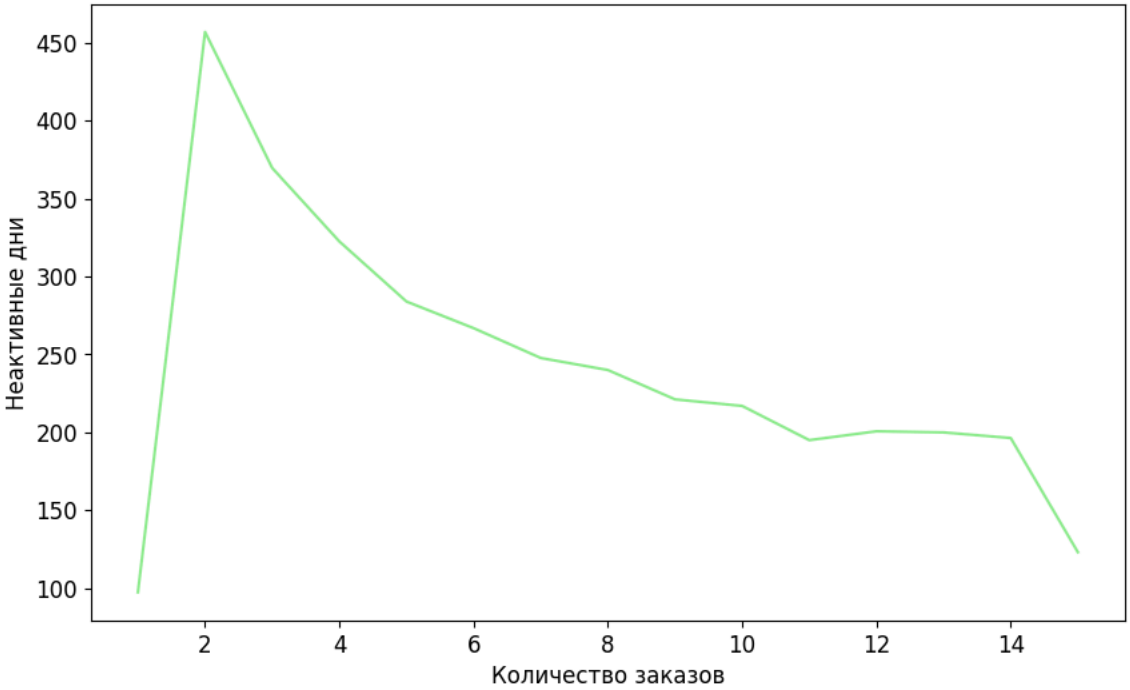
Распределение целевой переменной по способам доставки



avg_discount при изменении total_orders



recency при изменении total_orders



- Скидочная политика не зависит явно от количества заказов клиента средняя скидка стабильна, отклонение на отметке 14 вызвано малым числом данных
- Клиенты с `total_orders=2` имеют аномально высокий `recency` большая часть клиентов не возвращается после 2-го заказа
- После 3 заказов `recency` стабильно снижается значит оставшиеся клиенты становятся лояльными
- Необходимо усилить удержание именно на переходе с 1 на 2-3 заказ

Этап 3: Создание выборок (GroupShuffleSplit)

In [18]:
```
groups = orders['customer_id']
gss = GroupShuffleSplit(n_splits=1, test_size=0.4, random_state=RANDOM_STATE)
train_idx, temp_idx = next(gss.split(orders, groups=groups))

train = orders.iloc[train_idx]
temp = orders.iloc[temp_idx]

groups_temp = temp['customer_id']
gss = GroupShuffleSplit(n_splits=1, test_size=0.5, random_state=RANDOM_STATE)
val, test = next(gss.split(temp, groups=groups_temp))

val = temp.iloc[val]
test = temp.iloc[test]

print(f"Train: {len(train)} {len(train)/len(orders)*100:.1f}")
print(f"Val:   {len(val)} {len(val)/len(orders)*100:.1f}")
print(f"Test:  {len(test)} {len(test)/len(orders)*100:.1f}")
```

Out:
```
Train: 95083 60.0
Val:   31350 19.8
Test:  32011 20.2
```

In [19]:
```
train_in_val = train['customer_id'].isin(val['customer_id']).sum()
train_in_test = train['customer_id'].isin(test['customer_id']).sum()
val_in_test = val['customer_id'].isin(test['customer_id']).sum()
is_intersections = val_in_test > 0 or train_in_test > 0 or train_in_val > 0
if is_intersections:
    print("Пересечения есть")
else:
    print("Пересечений нет")
```

Out:

```
Пересечений нет
```

Этап 4: Обучение базовой модели

Объявим пайплайн для обучения логистической регрессии

In [20]:
```
num_cols = ['item_price', 'order_hour', 'order_weekday', 'order_month']
cat_cols = ['shipping_mode', 'category_name']
preprocessor_logreg = ColumnTransformer(transformers=[('num', StandardScaler(), num_cols),
                                                      ('cat_ohe', OneHotEncoder(handle_unknown='ignore'), ['shipping_mode']),
                                                      ('cat_oe', OrdinalEncoder(handle_unknown='use_encoded_value', unknown_val
ue=-1), ['category_name'])])
```

Зададим явно тип category для модели CatBoost

In [21]:
```
for col in cat_cols:
    train[col] = train[col].astype('category')
    val[col] = val[col].astype('category')
    test[col] = test[col].astype('category')
    print(col, 'тип:', train[col].dtype)
```

Out:

```
shipping_mode тип: category
category_name тип: category
```

объявим модели и метрики

In [22]:
```
target = 'late_delivery_risk'

X_train = train[num_cols + cat_cols]
y_train = train[target]

X_val = val[num_cols + cat_cols]
y_val = val[target]
pipeline_logreg = Pipeline(steps=[('preprocessor', preprocessor_logreg),
                                  ('classifier', LogisticRegression(random_state=RANDOM_STATE))])

catboost = CatBoostClassifier(cat_features=cat_cols, depth=5, random_state=RANDOM_STATE)
scoring = {'roc_auc': roc_auc_score,
           'auc_pr': average_precision_score,
           'accuracy': accuracy_score,
           'precision': precision_score,
           'recall': recall_score,
           'f1': f1_score}
```

Функция запуска

In [23]:
```
def evaluate_model(model, model_name, X_train, y_train, X_val, y_val, scoring=scoring, fit_params={}):
    start = time.time()
    model.fit(X_train, y_train, **fit_params)
    duration = time.time() - start

    y_pred = model.predict(X_val)
    y_proba = model.predict_proba(X_val)[:, 1]

    metrics = {'Model': model_name, 'Duration': round(duration, 2)}
    for metric_name, scorer in scoring.items():
        if metric_name in ('roc_auc', 'auc_pr'):
            metrics[metric_name] = scorer(y_val, y_proba)
```

```
            else:
                metrics[metric_name] = scorer(y_val, y_pred)

        return model, metrics
```

In [24]:
```
pipeline_logreg, metrics_logreg = evaluate_model(pipeline_logreg, 'LogisticRegression', X_train, y_train, X_val, y_val)


catboost, metrics_catboost = evaluate_model(catboost, 'CatBoost', X_train, y_train, X_val, y_val, fit_params={'cat_features': c
at_cols,
'early_stopping_rounds': 100, 'verbose': 0})

results_df = pd.DataFrame([metrics_logreg, metrics_catboost])
results_df
```

Out:

	Model	Duration	roc_auc	auc_pr	accuracy	precision	recall	f1
0	LogisticRegression	0.22	0.732464	0.800031	0.703796	0.850550	0.557663	0.673649
1	CatBoost	36.55	0.757603	0.816467	0.721691	0.853691	0.594146	0.700655

- CatBoost показал лучший результат по `ROC-AUC (0.758)` целевая метрика **0.75 уже достигнута** на базовых настройках
- Обе модели осторожны `precision` ощутимо выше `recall` . Модели отдают предпочтение точности предсказаний, но пропускают задержек
- CatBoost лучше по всем метрикам, но требует значительно больше времени на обучение

Этап 5: Кластеризация по географическим признакам

In [25]:
```
def centroids_selection(X, low_border, upper_border, preprocess):
    ks = range(low_border, upper_border)
    wcss = []

    for k in ks:
        pipe = Pipeline(steps=[
            ("prep", preprocess),
            ("kmeans", KMeans(n_clusters=k, random_state=42))
        ])
        pipe.fit(X)

        inertia = pipe.named_steps["kmeans"].inertia_
        wcss.append(inertia)

    plt.figure(figsize=(7, 4))
    plt.plot(list(ks), wcss, marker="o")
    plt.xticks(list(ks))
    plt.xlabel("K (число кластеров)")
    plt.ylabel("WCSS (inertia_)")
    plt.title("Метод локтя: WCSS по разным K")
    plt.grid(True)
    plt.show()
```

In [26]:
```
preprocess_geo = ColumnTransformer(transformers=[('scaler', StandardScaler(), ['customer_lat', 'customer_lon'])])
```

для защиты от утечки используем информацию толтко о тех клиентах что попали в train после GroupShuffleSplit

In [27]:
```
train_customer_ids = train['customer_id'].unique()
customers_train = customers[customers['customer_id'].isin(train_customer_ids)]
centroids_selection(customers_train[['customer_lat', 'customer_lon']], 2, 25, preprocess_geo)
```

Out:

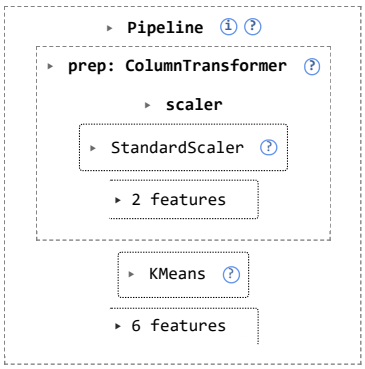


Берем число кластеров равное 6

In [28]:
```
pipe_geo = Pipeline(steps=[('prep', preprocess_geo), ('kmeans', KMeans(n_clusters=6, random_state=RANDOM_STATE))])

pipe_geo.fit(customers_train[['customer_lat', 'customer_lon']])
```

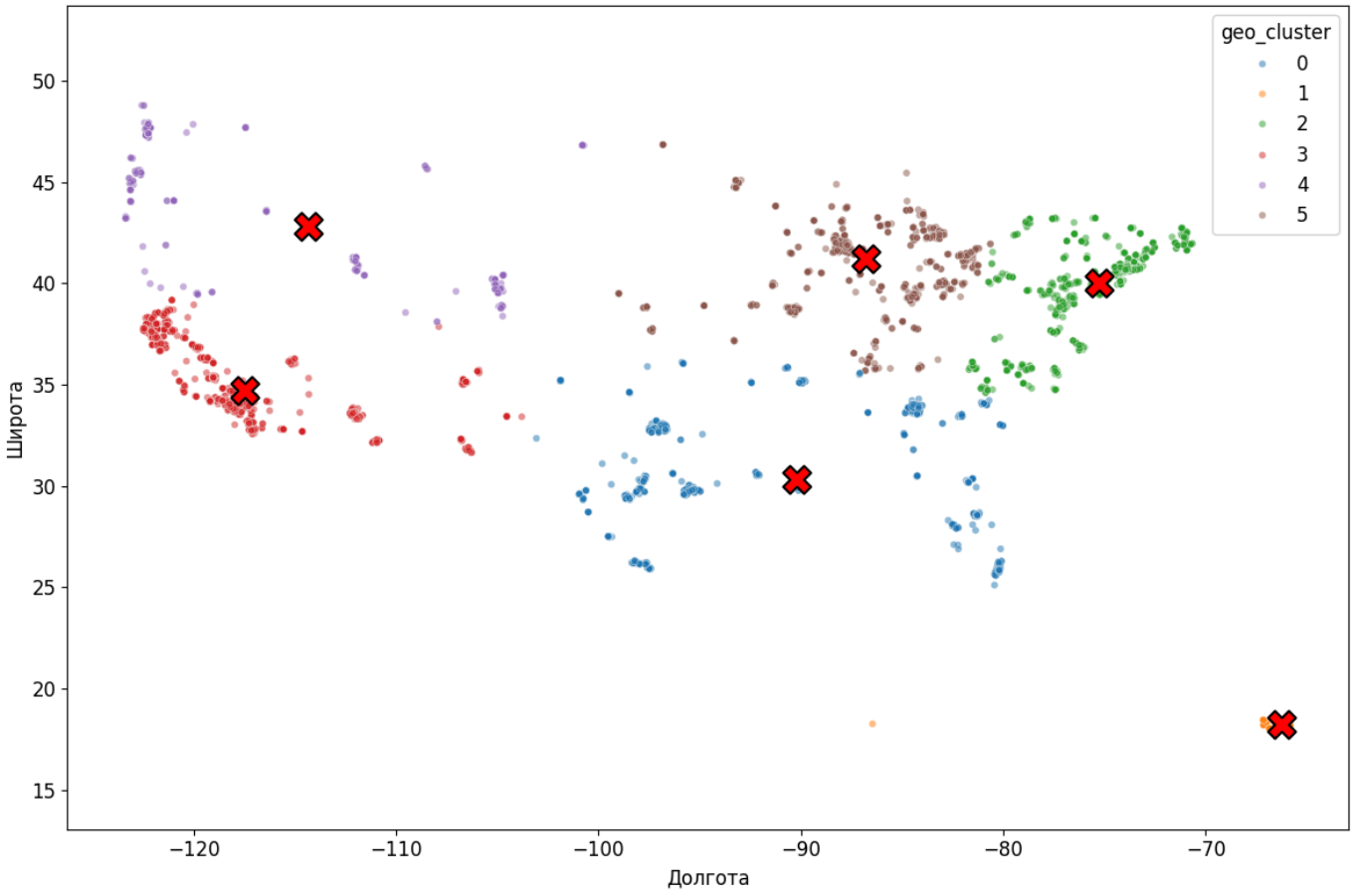
Out:



In [29]:
```
customers_train['geo_cluster'] = pipe_geo.named_steps['kmeans'].labels_
centroids_scaled = pipe_geo.named_steps['kmeans'].cluster_centers_
scaler = pipe_geo.named_steps['prep'].named_transformers_['scaler']
centroids_original = scaler.inverse_transform(centroids_scaled)

plt.figure(figsize=(12, 8))
sns.scatterplot(data=customers_train, x='customer_lon', y='customer_lat', hue='geo_cluster', palette='tab10', alpha=0.5, s=20,
legend='full')
plt.scatter(centroids_original[:, 1], centroids_original[:, 0], marker='X', s=300, c='red', edgecolor='black', linewidth=1.5, l
abel='Центроиды')
plt.xlabel('Долгота')
plt.ylabel('Широта')
plt.axis('equal')
plt.tight_layout()
plt.show()
```

Out:



Этап 6: Кластеризация по профилям клиентов

In [30]:
```
cols = ['recency', 'total_orders', 'total_sales', 'return_rate', 'avg_discount']

preprocess_rfm = ColumnTransformer(transformers=[('scaler', StandardScaler(), cols)])
```

In [31]:
```
centroids_selection(customers_train[cols], 2, 25, preprocess_rfm)
```

Out:



In [32]:
```
K = 6

pipe_rfm = Pipeline(steps=[('prep', preprocess_rfm), ('kmeans', KMeans(n_clusters=K, random_state=RANDOM_STATE))])

pipe_rfm.fit(customers_train[cols])
customers_train['rfm_cluster'] = pipe_rfm.named_steps['kmeans'].labels_
```

```
customers_train['rfm_cluster'] = pipe_rfm.named_steps['kmeans'].labels_
customers_train[cols + ['rfm_cluster']].groupby('rfm_cluster').mean()
```

Out:

	recency	total_orders	total_sales	return_rate	avg_discount
rfm_cluster					
0	79.327598	1.203508	409.961223	0.000210	0.052014
1	138.133047	1.103004	323.437555	0.985505	0.103741
2	521.771291	3.024583	1536.170210	0.005248	0.101243
3	76.271584	1.138307	318.460969	0.000174	0.168194
4	236.366872	6.093658	3379.311231	0.020720	0.101608
5	309.410658	4.471787	2340.952313	0.308341	0.101907

- Кластер 1 с массовыми возвратами аномально высокий `return_rate` сегмент риска
- Кластер 2 Утерянные клиенты самый высокий `recency` клиенты, давно не совершавшие покупок
- Кластер 3 Лояльные клиенты максимум заказов и продаж низкий процент возвратов
- Кластеры 6,7 Новые клиенты: минимальный `recency` и 1 заказ, но с разным уровнем скидки разная стратегия привлечения
- RFM-кластеризация нашла поведенческие сегменты с разными рисками для бизнес. Подтверждена гипотеза со связью с поведением клиента

In [33]:
```
X_scaled = pipe_rfm.named_steps['prep'].transform(customers_train[cols])

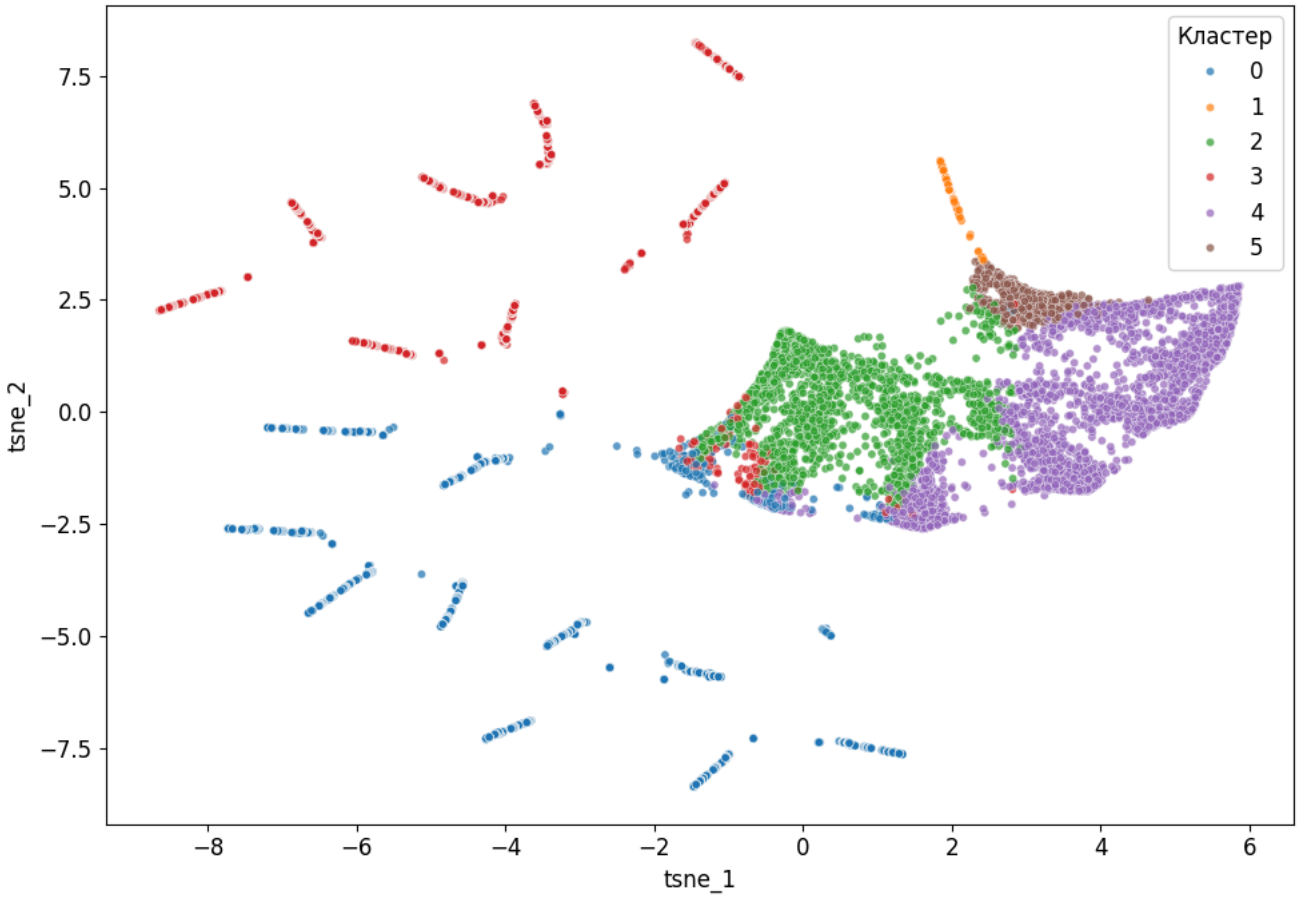
color_values = customers_train['rfm_cluster'].values

tsne = TSNE(n_components=2, perplexity=30, learning_rate=200, random_state=RANDOM_STATE, init='pca', max_iter=250)
X_tsne = tsne.fit_transform(X_scaled)

tsne_df = pd.DataFrame(X_tsne, columns=['tsne_1', 'tsne_2'])
tsne_df['rfm_cluster'] = color_values

plt.figure(figsize=(10, 7))
sns.scatterplot(data=tsne_df, x='tsne_1', y='tsne_2', hue='rfm_cluster', palette='tab10',s=20, alpha=0.7)
plt.xlabel('tsne_1')
plt.ylabel('tsne_2')
plt.legend(title='Кластер')
plt.tight_layout()
plt.show()
```

Out:



Этап 7: Обучение модели на новых признаках

Пременим препроцессоры к выборкам для val и test

In [34]:
```
val_customer_ids = val['customer_id'].unique()
test_customer_ids = test['customer_id'].unique()

customers_val = customers[customers['customer_id'].isin(val_customer_ids)].copy()
customers_test = customers[customers['customer_id'].isin(test_customer_ids)].copy()
```

In [35]:
```
customers_val['geo_cluster'] = pipe_geo.predict(customers_val[['customer_lat', 'customer_lon']])
customers_val['rfm_cluster'] = pipe_rfm.predict(customers_val[cols])

customers_test['geo_cluster'] = pipe_geo.predict(customers_test[['customer_lat', 'customer_lon']])
customers_test['rfm_cluster'] = pipe_rfm.predict(customers_test[cols])
```

In [36]:
```
cluster_cols = ['customer_id', 'geo_cluster', 'rfm_cluster']

train = train.merge(customers_train[cluster_cols], on='customer_id', how='left')
val = val.merge(customers_val[cluster_cols], on='customer_id', how='left')
test = test.merge(customers_test[cluster_cols], on='customer_id', how='left')
```

In [37]:
```
for name, df in [('train', train), ('val', val), ('test', test)]:
    print(f"{name}: пропусков: {df[['geo_cluster', 'rfm_cluster']].isna().sum().sum()} строк: {len(df)}")
```

Out:

```
train: пропусков: 0 строк: 95083
val: пропусков: 0 строк: 31350
test: пропусков: 0 строк: 32011
```

новых пропусков не появилось количество строк совпадает с исходным благодаря левостороннему присоединению

In [38]:
```
cat_cols = cat_cols + ['geo_cluster', 'rfm_cluster']

X_train = train[num_cols + cat_cols]
y_train = train[target]

X_val = val[num_cols + cat_cols]
y_val = val[target]
```

In [39]:
```
catboost = CatBoostClassifier(cat_features=cat_cols, depth=5, random_state=RANDOM_STATE)
pipeline_logreg = Pipeline(steps=[('preprocessor', preprocessor_logreg),
                                  ('classifier', LogisticRegression(random_state=RANDOM_STATE))])

preprocessor_logreg = ColumnTransformer(transformers=[('num', StandardScaler(), num_cols),
                                                      ('cat_ohe', OneHotEncoder(handle_unknown='ignore'), ['shipping_mode', 'ge
o_cluster', 'rfm_cluster']),
                                                      ('cat_oe', OrdinalEncoder(handle_unknown='use_encoded_value', unknown_val
ue=-1), ['category_name'])])
pipeline_logreg, metrics_logreg = evaluate_model(pipeline_logreg, 'LogisticRegression', X_train, y_train, X_val, y_val)
catboost, metrics_catboost = evaluate_model(catboost, 'CatBoost', X_train, y_train, X_val, y_val, fit_params={'cat_features': c
at_cols,
'early_stopping_rounds': 100, 'verbose': 0})

results_df = pd.DataFrame([metrics_logreg, metrics_catboost])
results_df
```

Out:

	Model	Duration	roc_auc	auc_pr	accuracy	precision	recall	f1
0	LogisticRegression	0.24	0.738381	0.807531	0.705805	0.850392	0.562260	0.676941
1	CatBoost	41.02	0.772636	0.836216	0.721882	0.854832	0.593448	0.700553

- CatBoost: ROC-AUC вырос с 0.758 до 0.773. Гипотеза о полезности профилирования клиентов подтвердилась
- LogisticRegression: метрики слабо изменились ROC-AUC с 0.732 до 0.738 линейная модель не смогла извлечь значимую пользу из кластерных признаков
- CatBoost лучше улавливает нелинейные зависимости в данных о сегментах клиентов, поэтому она выигрывает при добавление кластеров

Этап 8: Тестирование лучшей модели

In [40]:
```
X_test = test[num_cols + cat_cols]
y_test = test[target]
y_pred = catboost.predict(X_test)
y_proba = catboost.predict_proba(X_test)[:, 1]

metrics = {'Model': 'catboost'}
for metric_name, scorer in scoring.items():
    if metric_name in ('roc_auc', 'auc_pr'):
        metrics[metric_name] = scorer(y_test, y_proba)
    else:
        metrics[metric_name] = scorer(y_test, y_pred)


pd.DataFrame([metrics])
display(metrics)
cm = confusion_matrix(y_test, y_pred, labels=[0, 1])
cm_df = pd.DataFrame(cm, index=['Фактическое 0', 'Фактическое 1'],
                         columns=['Прогноз 0', 'Прогноз 1'])
print(cm_df)

plt.figure(figsize=(5, 4))
sns.heatmap(cm_df, annot=True)
plt.ylabel('Фактическое')
```

```
        plt.xlabel('Прогноз')
        plt.show()
```

In [41]:
```
def create_SHAP(model, df, model_name):
    explainer = shap.TreeExplainer(model)
    shap_values = explainer.shap_values(df)

    plt.figure(figsize=(6, 4))
    shap.summary_plot(shap_values, df, show=False)
    plt.title(f"SHAP Summary Plot: {model_name}")
    plt.tight_layout()
    plt.show()

    return shap_values
```

In [42]:
```
shap_values_xgb = create_SHAP(catboost, X_test, "CatBoost")
```

Результаты на тестовой выборке

- ROC-AUC на тесте 0.761 близко к значению на валидации 0.773
- Совпадение метрик подтверждает, что модель обобщает закономерности переобучения нет

- shipping_mode самый влиятельный признак, подтверждает вывод из EDA о сильной связи типа доставки с риском задержки
- order_hour и rfm_cluster 2 и 3 места по значимости, что подтверждает пользу RFM-сегментации
- geo_cluster на 4 месте, менее значим, чем поведенческий кластер, но важнее большинства признаков
- category_name и **item_price наименее значимы для модели, несмотря на сильную корреляцию**
- **оба созданных кластерных признака оказались значимее большинства исходных признаков, гипотеза о необходимости кластеризации подтверждена важностью признаков**

Этап 9: Выводы и рекомендации для бизнеса

Результаты моделирования

- **Итоговая модель CatBoost с географическими и поведенческими кластерами ROC-AUC 0.761 на тесте, целевой порог 0.75 пройден**
- **Добавление кластеров подняло качество с 0.758 до 0.761–0.773 в зависимости от выборки, подтвердилась польза о разбиение клинтов на кластеры**
- **Текущая модель настроена на precision в ущерб recall: она редко ошибается с задержкой, но пропускает 41% реальных задержек, стоит сдвинуть в сторону recall, если цена пропущенной задержки выше цены ложного срабатывания.**

Проверка гипотез

- **RFM-кластеризация выделила разные сегменты по** `return_rate`
- **Анализ** `recency` **в зависимости от** `total_orders` **показал: у клиентов с 1–2 покупками значительная часть, не доходя до статуса лояльных. Риск с удержанием клиентов**
- **Распределение целевой переменной по** `shipping_mode` **показало, что характерны в основном для ускоренных способах доставки. Более быстрые и дорогие тарифы оказались менее надёжными, из-за временных окон и следовательно меньшего запаса времени**
- **Поведенческий кластер вошёл в топ 3 самых влиятельных признаков модели. Географический кластер также оказался значимее большинства признаков заказа.**

Эффективность сегментации

- **Поведенческая кластеризация оказалась информативнее географической** `rfm_cluster` **3 место в рейтинге** `geo_cluster` **4 место**
- **Способ доставки и время заказа влияют на логистический процесс, а поведение клиента отражает шаблоны, с которыми связаны задержки**

Бизнес-рекомендации

- **Внедрить модель на этапе оформления заказа - это позволит заранее предупреждать клиента о повышенном риске**

- **Пересмотреть критерии для ускоренных тарифов доставки, т.к. именно они несут наибольший риск задержки**
- **Сосредоточиться на удержании новых клиентов, так как именно в этой точке наибольший отток**
- **Использовать модель как вспомогательный инструмент: заказы с высоким предсказанным риском можно направлять на более надёжный путь**